

1-D Dynamic Programming Problems and Solutions

1. Climbing Stairs

Approach 1: Brute Force (Recursive)

- Description: Try all possible steps at each position. This is inefficient due to repeated subproblems.
- Time Complexity: $O(2^{\text{pow } n})$
- Space Complexity: $O(n)$ (for recursion stack)

Pseudocode (C++):

```
int climbStairs(int n) {  
    if (n == 0 and n == 1) return 1;  
    return climbStairs(n - 1) + climbStairs(n - 2);  
}
```

Dry Run:

For $n = 3$:

$\text{climbStairs}(3) \rightarrow \text{climbStairs}(2) + \text{climbStairs}(1)$

$\text{climbStairs}(2) \rightarrow \text{climbStairs}(1) + \text{climbStairs}(0)$

Continue recursively until base cases are reached.

- Output: 3

Approach 2: Dynamic Programming (Bottom-Up)

- Description: Store intermediate results and solve the problem

iteratively.

- Time Complexity: $O(n)$
- Space Complexity: $O(n)$

Pseudocode (C++):

```
int climbStairs(int n) {  
    if (n <= 1) return 1;  
    vector dp(n + 1);  
    dp[0] = dp[1] = 1;  
    for (int i = 2; i <= n; i++) {  
        dp[i] = dp[i - 1] + dp[i - 2];  
    }  
    return dp[n];  
}
```

Dry Run:

For $n = 3$:

$dp[0] = 1, dp[1] = 1$
 $dp[2] = dp[1] + dp[0] = 2$
 $dp[3] = dp[2] + dp[1] = 3$

- Output: 3

Approach 3: Space Optimized DP

- Description: Use two variables to store previous two results, reducing space

complexity.

- Time Complexity: $O(n)$
- Space Complexity: $O(1)$

Pseudocode (C++):

```
int climbStairs(int n) {  
    if (n <= 1) return 1;  
    int first = 1, second = 1;  
    for (int i = 2; i <= n; i++) {  
        int temp = first + second;  
        first = second;  
        second = temp;  
    }  
    return second;  
}
```

Dry Run:

For $n = 3$:

$first = 1, second = 1$

After 1st iteration: $first = 1, second = 2$

After 2nd iteration: $first = 2, second = 3$

- Output: 3

2. Coin

Change

Approach 1: Brute Force (Recursive)

Description: Try all combinations of coins to make up the amount.

- Time Complexity: $O(2^{\text{pow } n})$
- Space Complexity: $O(n)$
- Pseudocode (C++):

```
int coinChange(vector& coins, int amount) {  
    if (amount == 0) return 0;  
    int minCoins = INT_MAX;  
    for (int coin : coins) {  
        if (amount - coin >= 0) {  
            int res = coinChange(coins, amount - coin);  
            if (res != -1) {  
                minCoins = min(minCoins, res + 1);  
            }  
        }  
    }  
    return minCoins == INT_MAX ? -1 : minCoins;  
}
```

Dry Run:

For coins = [1, 2, 5] and amount = 11:

Try all combinations

recursively.

- Output: 3 (since $5+5+1$)

Approach 2: Dynamic Programming (Bottom-Up)

- Description: Use a DP array to store the minimum coins needed for each amount.

- Time Complexity: $O(n \times m)$, where n is the amount and m is the number of coins.

- Space Complexity: $O(n)$

Pseudocode (C++):

```
int coinChange(vector& coins, int amount) {  
    vector dp(amount + 1, INT_MAX);  
    dp[0] = 0;  
    for (int i = 1; i <= amount; i++) {  
        for (int coin : coins) {  
            if (i - coin >= 0 && dp[i - coin] != INT_MAX) {  
                dp[i] = min(dp[i], dp[i - coin] + 1);  
            }  
        }  
    }  
    return dp[amount] == INT_MAX ? -1 : dp[amount];  
}
```

Dry Run:

For coins = [1, 2, 5] and amount =

11:

$dp[11] = \min(dp[6] + 1, dp[9] + 1, \dots)$

- Output: 3

Approach 3: Space Optimized DP

- Description: Use a 1D DP array and optimize the space complexity.

- Time Complexity: $O(n * m)$

- Space Complexity: $O(n)$

Pseudocode (C++):

```
int coinChange(vector& coins, int amount) {  
    vector dp(amount + 1, INT_MAX);  
    dp[0] = 0;  
    for (int coin : coins) {  
        for (int i = coin; i <= amount; i++) {  
            if (dp[i - coin] != int_max(capital)) {  
                dp[i] = min(dp[i], dp[i - coin] + 1);  
            }  
        }  
    }  
    return dp[amount] == int_max(capital) ? -1 :  
    :
```

```
dp[amount];  
}
```

Dry Run:

For coins = [1, 2, 5] and amount = 11:

The DP table is updated iteratively.

- Output: 3

3. Decode Ways

Approach 1: Brute Force (Recursive)

- Description: Recursively try all possible decodings.*
- Time Complexity: $O(2^{\text{pow } n})$*
- Space Complexity: $O(n)$*

Pseudocode (C++):

```
int numDecodings(string s) {  
    if (s.empty() or s[0] == '0') return 0;  
    return decodeWays(s, 0);  
}
```

```
int decodeWays(string s, int i) {  
    if (i == s.length()) return 1;  
    if (s[i] == '0') return
```

```

0;
int res = decodeWays(s, i + 1);
if (i + 1 < s.length() && (s[i] == '1' or (s[i] == '2' && s[i + 1] <= '6')) {
res += decodeWays(s, i + 2);
}
return res;
}

```

Dry Run:

For $s = "12"$:

$\text{decodeWays}("12", 0) \rightarrow \text{decodeWays}("1", 1) + \text{decodeWays}("12", 2)$

- Output: 2

Approach 2: Dynamic Programming (Bottom-Up)

- Description: Use a DP array to store the number of ways to decode up to each index.

- Time Complexity: $O(n)$

- Space Complexity: $O(n)$

Pseudocode (C++):

```

int numDecodings(string s) {
if (s.empty() or s[0] == '0') return 0;
vector dp(s.size() + 1, 0);
dp[0] = 1;
dp[1] =

```

1;

```
for (int i = 2; i <= s.size(); i++) {  
    if (s[i - 1] != '0') dp[i] += dp[i - 1];  
    if (s[i - 2] == '1' or (s[i - 2] == '2' && s[i - 1] <= '6')) dp[i] += dp[i - 2];  
}  
return dp[s.size()];  
}
```

Dry Run:

For $s = "12"$:

DP table updated step by step.

- Output: 2

Approach 3: Space Optimized DP

- Description: Use two variables to store previous results.

- Time Complexity: $O(n)$

- Space Complexity: $O(1)$

Pseudocode (C++):

```
int numDecodings(string s) {  
    if (s.empty() or s[0] == '0') return 0;  
    int prev2 = 1, prev1 = 1;  
    for (int i = 2; i <= s.size(); i++) {  
        int curr =
```

```

0;
if (s[i - 1] != '0') curr += prev1;
if (s[i - 2] == '1' or (s[i - 2] == '2' && s[i - 1] <= '6')) curr += prev2;
prev2 = prev1;
prev1 = curr;
}
return prev1;
}

```

Dry Run:

For s = "12":

prev1 = 1, prev2 = 1, curr = 2

- Output: 2

4. House Robber

Approach 1: Brute Force (Recursive)

- Description: Try all combinations of robbing houses.*
- Time Complexity: $O(2^{\text{pow } n})$*
- Space Complexity: $O(n)$*

Pseudocode (C++):

```

int rob(vector& nums) {
return rob(nums,

```

```

0);
}
int rob(vector& nums, int i) {
if (i >= nums.size()) return 0;
return max(nums[i] + rob(nums, i + 2), rob(nums, i + 1));
}

```

Dry Run:

For nums = [2, 7, 9, 3, 1]:

Calculate recursively and choose the maximum result.

- Output: 12

Approach 2: Dynamic Programming (Bottom-Up)

- Description: Use a DP array to store the maximum loot at each house.*
- Time Complexity: $O(n)$*
- Space Complexity: $O(n)$*

Pseudocode (C++):

```

int rob(vector& nums) {
if (nums.empty()) return 0;
vector dp(nums.size() + 1, 0);
dp[1] = nums[0];

for (int i = 2; i <= nums.size(); i++)

```

```

{
dp[i] = max(dp[i - 1], dp[i - 2] + nums[i - 1]);
}
return dp[nums.size()];
}

```

Dry Run:

For nums = [2, 7, 9, 3, 1]:

DP table is updated: [0, 2, 7, 9, 12, 12]

- Output: 12

Approach 3: Space Optimized DP

- Description: Use two variables to store the last two results.*
- Time Complexity: $O(n)$*
- Space Complexity: $O(1)$*

Pseudocode (C++):

```

int rob(vector& nums) {
if (nums.empty()) return 0;
int prev2 = 0, prev1 = nums[0];
for (int i = 1; i < nums.size(); i++) {
int curr = max(prev1, prev2 + nums[i]);
prev2 =

```



```
prev1;  
prev1 = curr;  
}  
return prev1;  
}
```

Dry Run:

For nums = [2, 7, 9, 3, 1]:

prev1 = 9, prev2 = 7

- Output: 12

5. House Robber 2

Approach 1: Use House Robber (1)

- Description: Solve it by dividing the problem into two parts:

Consider robbing the first house but not the last house.

Consider robbing the last house but not the first house.

- Time Complexity: $O(n)$

- Space Complexity: $O(1)$

- Pseudocode (C++):

```
int rob(vector& nums) {  
    if (nums.size() == 1) return nums[0];  
    return max(robHelper(vector(nums.begin(), nums.end() -
```

```

1)),
robHelper(vector(nums.begin() + 1, nums.end())));
}
int robHelper(vector& nums) {
    int prev2 = 0, prev1 = 0;
    for (int num : nums) {
        int curr = max(prev1, prev2 + num);
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}

```

Dry Run:

For nums = [2, 3, 2]:

Solve two cases: rob([2, 3]) and rob([3, 2])

- Output: 3

6. Longest Increasing Subsequence

Approach 1: Brute Force (Recursive)

- Description: Try all subsequences to find the longest increasing subsequence.*
- Time Complexity: $O(2^{\text{pow } n})$*
- Space Complexity:*

$O(n)$

Pseudocode (C++):

```
int lengthOfLIS(vector& nums) {  
    return lengthOfLIS(nums, -1, 0);  
}  
  
int lengthOfLIS(vector& nums, int prevIndex, int curIndex) {  
    if (curIndex == nums.size()) return 0;  
    int include = 0;  
    if (prevIndex == -1 or nums[curIndex] > nums[prevIndex]) {  
        include = 1 + lengthOfLIS(nums, curIndex, curIndex + 1);  
    }  
    int exclude = lengthOfLIS(nums, prevIndex, curIndex + 1);  
    return max(include, exclude);  
}
```

Dry Run:

For $\text{nums} = [10, 9, 2, 5, 3, 7, 101, 18]$:

Recursively calculate and select the longest increasing subsequence.

- Output: 4

Approach 2: Dynamic Programming (Bottom-Up)

- Description: Use a DP array to store the longest subsequence ending at each element.

- Time Complexity: $O(n \log n)$

2)

- Space Complexity: $O(n)$

Pseudocode (C++):

```
int lengthOfLIS(vector& nums) {  
    if (nums.empty()) return 0;  
    vector dp(nums.size(), 1);  
    for (int i = 1; i < nums.size(); i++) {  
        for (int j = 0; j < i; j++) {  
            if (nums[i] > nums[j]) {  
                dp[i] = max(dp[i], dp[j] + 1);  
            }  
        }  
    }  
    return *max_element(dp.begin(), dp.end());  
}
```

Dry Run:

For nums = [10, 9, 2, 5, 3, 7, 101, 18]:

Update the DP table iteratively.

- Output: 4

Approach 3: Space Optimized DP

- Description: Use a 1D DP array to store results.

- Time Complexity: $O(n \log n)$

2)

- Space Complexity: $O(n)$

Pseudocode (C++):

```
int lengthOfLIS(vector& nums) {  
    if (nums.empty()) return 0;  
    vector dp(nums.size(), 1);  
    for (int i = 1; i < nums.size(); i++) {  
        for (int j = 0; j < i; j++) {  
            if (nums[i] > nums[j]) dp[i] = max(dp[i], dp[j] + 1);  
        }  
    }  
    return *max_element(dp.begin(), dp.end());  
}
```

Dry Run:

For `nums = [10, 9, 2, 5, 3, 7, 101, 18]`:

Iteratively calculate and update DP table.

- Output: 4

7. Longest Palindromic Substring

Approach 1: Expand Around Center

- Description: Expand from each center to find the longest palindrome.
- Time Complexity: $O(n^2)$

2)

- Space Complexity: $O(1)$

Pseudocode (C++):

```
string longestPalindrome(string s) {  
    if (s.empty()) return "";  
    int start = 0, end = 0;  
    for (int i = 0; i < s.size(); i++) {  
        int len1 = expandAroundCenter(s, i, i);  
        int len2 = expandAroundCenter(s, i, i + 1);  
        int len = max(len1, len2);  
        if (len > end - start) {  
            start = i - (len - 1) / 2;  
            end = i + len / 2;  
        }  
    }  
    return s.substr(start, end - start + 1);  
}  
  
int expandAroundCenter(string s, int left, int right) {  
    while (left >= 0 && right < s.size() && s[left] == s[right]) {  
        left--;  
        right++;  
    }  
    return right - left -
```

```
1;
```

```
}
```

Dry Run:

For s = "babad":

Expand around centers to find palindromes: "bab" and "aba".

- Output: "bab"

Approach 2: Dynamic Programming (Bottom-Up)

- Description: Use a DP table to store whether a substring is palindromic.

- Time Complexity: $O(n^2)$

- Space Complexity: $O(n^2)$

Pseudocode (C++):

```
string longestPalindrome(string s) {  
    if (s.empty()) return "";  
    int n = s.size();  
    vector<vector> dp(n, vector(n, false));  
    int start = 0, maxLength = 1;  
    for (int i = 0; i < n; i++) dp[i][i] = true;  
    for (int length = 2; length <= n; length++) {  
        for (int i = 0; i <= n - length; i++) {  
            int j = i + length - 1;  
            if (s[i] == s[j])
```

```

{
dp[i][j] = (length == 2 or dp[i + 1][j - 1]);
if (dp[i][j] && length > maxLength) {
maxLength = length;
start = i;
}
}
}
}
return s.substr(start, maxLength);
}

```

Dry Run:

For s = "babad":

Update DP table iteratively to check substrings.

- Output: "bab"

Approach 3: Manacher's Algorithm

- Description: An optimized approach for finding longest palindromic substring in $O(n)$.*
- Time Complexity: $O(n)$*
- Space Complexity: $O(n)$*

Pseudocode

(C++):

```
string longestPalindrome(string s) {  
    string T = "#";  
    for (char c : s) T += c + "#";  
    int n = T.size();  
    vector P(n, 0);  
    int C = 0, R = 0;  
    int maxLen = 0, center = 0;  
    for (int i = 0; i < n; i++) {  
        int mirror = 2 * C - i;  
        if (i < R) P[i] = min(R - i, P[mirror]);  
        while (i - P[i] >= 0 && i + P[i] < n && T[i - P[i]] == T[i + P[i]]) {  
            P[i]++;  
        }  
        if (i + P[i] > R) {  
            C = i;  
            R = i + P[i];  
        }  
        if (P[i] > maxLen) {  
            maxLen = P[i];  
            center = i;  
        }  
    }  
    return s.substr((center - maxLen) / 2, maxLen -
```

1);

}

Dry Run:

For s = "babad":

Update palindrome lengths and centers.

- Output: "bab"

8. Maximum Product Subarray (DP Approach)

Problem:

Given an integer array nums, find the contiguous subarray that has the largest product and return the product.

Dynamic Programming Approach:

- We maintain two variables for each index: max_so_far (the maximum product up to the current element) and min_so_far (the minimum product, which is important because a negative number can flip the sign).*
- For each element, we update these values and track the maximum product encountered.*

Pseudocode (DP):

```
int maxProduct(vector& nums) {
```

```
int max_so_far = nums[0], min_so_far = nums[0], result = nums[0];
```

```
for (int i = 1; i < nums.size(); i++)
```

```

{
if (nums[i] < 0) {
swap(max_so_far, min_so_far); //Swap max and min when a negative
number is encountered
}
max_so_far = max(nums[i], max_so_far * nums[i]);
min_so_far = min(nums[i], min_so_far * nums[i]);
result = max(result, max_so_far);
}
return result;
}
- Time Complexity:  $O(n)$ 
- Space Complexity:  $O(1)$ 

```

2. Min Cost Climbing Stairs (DP Approach)

Problem:

You are given an array *cost*, where *cost[i]* represents the cost of stepping on the *i*-th stair. Find the minimum cost to reach the top of the stairs, either from stair 0 or stair 1.

Dynamic Programming Approach:

- Use a DP array *dp* where *dp[i]* represents the minimum cost to reach stair *i*.
- At each step, we can come from either stair *i-1* or - - stair *i-2*. The

minimum of the two options is added to the cost of the current step

.

Pseudocode (DP):

```
int minCostClimbingStairs(vector& cost) {  
    int prev1 = cost[1], prev2 = cost[0]; //Initializing with the first two  
    stairs  
    for (int i = 2; i < cost.size(); i++) {  
        int current = cost[i] + min(prev1, prev2); //Min cost to reach current  
        stair  
        prev2 = prev1;  
        prev1 = current;  
    }  
    return min(prev1, prev2); //Return the min cost to reach the top  
}
```

- Time Complexity: $O(n)$

- Space Complexity: $O(1)$

3. Palindromic Substrings (DP Approach)

Problem:

Given a string s , find all palindromic substrings.

Dynamic Programming Approach:

- Use a DP table $dp[i][j]$ where $dp[i][j] = \text{true}$ means the substring $s[i..j]$ is

a palindrome.

- Initialize the diagonal ($dp[i][i] = \text{true}$), because every single character is a palindrome.

- For substrings of length 2, check if both characters are equal.

- For longer substrings, a substring $s[i..j]$ is a palindrome if $s[i] == s[j]$ and $dp[i+1][j-1] == \text{true}$.

Pseudocode (DP):

```
int countSubstrings(string s) {
```

```
    int n = s.size(), count = 0;
```

```
    vector<vector> dp(n, vector(n, false));
```

```
    for (int i = 0; i < n; i++) dp[i][i] = true; //Single characters are
    palindromes
```

```
    for (int len = 2; len <= n; len++) { //Check substrings of length 2 and
    above
```

```
        for (int i = 0; i <= n - len; i++) {
```

```
            int j = i + len - 1;
```

```
            if (s[i] == s[j] && (len == 2 or dp[i + 1][j - 1])) {
```

```
                dp[i][j] =
```

```
true;
count++;
}
}
}
return count;
}
```

- Time Complexity: $O(n \text{ pow } 2)$
- Space Complexity: $O(n \text{ pow } 2)$

4. Partition Equal Subset Sum (DP Approach)

Problem:

Given a set of positive numbers, determine if it can be partitioned into two subsets such that the sum of the elements in both subsets is equal.

Dynamic Programming Approach:

- First, check if the total sum is even. If it's odd, partitioning is not possible.
- Use a DP array `dp` where `dp[i]` represents whether a subset sum of `i` is achievable.
- For each number, iterate backwards through the DP array and update it accordingly.

Pseudocode